실제 목소리를 ,
가상 마을에서 실험하다

실제 댓글 •리뷰 → 자료 선별 → 3~4 인 토론 → 대화 흐름 시각화

핵심 주제

실제 텍스트와 가상 대화를 같은 기준으로 비교하는 AI 실험 도구

구현 기반

React · FastAPI · SQLite · OpenAI · Canvas

발표 기준

현재 구현 코드 검토본 · 2026.10.27

어떤 정보가 , 누구의 대화에서 달라지는가 ?

원문과 시뮬레이션을 연결해 관찰 가능한 실험으로 만든다 .

자료

YouTube 공개 댓글 ·답글 , K-HATERS 표본 , 사용자가 제공한 리뷰

관찰

인물별 발언과 이번 참고 자료 , 대화 단계 , AI 감정 ·확실성 분류

결과

선택한 원문과 가상 발화의 수치 · 관계망 ·히스토리 비교

CODE frontend/src/RealDataLab.jsx · frontend/src/DiscussionResults.jsx

수집에서 시각화까지 , 한 흐름으로

기존 마을 디자인을 유지하고 실제 데이터를 첫 화면에 배치했다 .

01 수집 ·가져오기

02 익명화 ·저장

03 정렬 ·최대 10 건 선택

04 3~4 인 토론

05 분석 ·비교 · 다운로드

수집 30 건 / 토론 10 건

미리보기 후 확인한 자료만 저장하고 , 선택 순서대로 토론 자료를 고정한다 .

단계 1~20 회

묶음 토론은 단계 수가 자료 수와 참여 인원 이상이어야 한다 .

CODE frontend/src/main.jsx · frontend/src/RealDataLab.jsx

실제 자료를 먼저 정리한다

정렬 •검색 •선별 후 , 자료 묶음을 마을로 전달한다 .



정렬

수집 순서 · 긴 문장 · AI 부정 감정 · 높은 별점

선별

문장 검색과 체크박스 / 최대 10 건

모드 선택

3 명 또는 4 명 / 4 번째 인물은 실행 전용 가상 자료 검토자

CODE frontend/src/RealDataLab.jsx

화면 ·실행 ·분석 ·저장을 분리한 구조

FastAPI 가 React 정적 빌드와 API 를 같은 Origin 에서 제공한다 .

React / Vite

FastAPI / Pydantic

Simulation + Analysis

SQLite / Snapshots

외부 입력

YouTube Data API v3 · Hugging Face 데이터 뷰어 · 브라우저 파일 읽기

실시간 경로

POST /api/runs/stream → SSE 이벤트 → React 상태 → Canvas· 말풍선

영속화 경로

완료 실행만 트랜잭션 저장 → 접근 키로 비공개 실행 조회

CODE Dockerfile · app/main.py

공식 API 와 연구 파일을 구분해 연결한다

자유로운 URL 크롤링 대신 고정된 수집 경로와 작은 호출 한도를 사용한다 .

YouTube 공식 API

commentThreads.list + comments.list / 실제 부모 - 답글 관계 유지 / 서버 전용 YOUTUBE_API_KEY

K-HATERS

humane-lab/K-HATERS / splits·rows 조회 / CC BY 4.0 출처 •변경 표시

KOLD 파일 가져오기

공식 JSON/JSONL 을 사용자가 확보 / 원본 주석자 ID 제거 / 서버 자동 재배포 없음

CODE app/services/collection.py · app/routers/collection.py

자료 ·대화 ·분석을 연결하는 데이터 계약

브라우저에서 가림 처리하고 서버에서도 다시 정리한다 .

```
X-Dataset-Key: <브라우저 생성 64 자리 hex>
→ 서버에는 owner hash 저장
→ dataset 과 연결 run 은 owner 로 조회
→ source_data.rows 에 선택 자료 스냅샷
```

DatasetRow

text · author · reply_to_id · rating · date · reference_labels

Source snapshot

선택한 row_id·text·analysis·origin 을 실행 당시 자료로 보존

Run / Event

speaker_id · listener_id · utterance · discussion_stage · source_row_ids · analysis

CODE app/routers/datasets.py · app/services/datasets.py

인물과 자료를 번갈아 배정한다

자료 요약 → 근거 점검 → 다른 관점 → 종합 정리

```
speaker = roster[(step - 1) % len(roster)]
listener = roster[step % len(roster)]
evidence = source["rows"][(step - 1) % len(source["rows"])]

# OpenAI 입력 : 전체 선택 자료 + 최근 4 개 발언
# 단계마다 대화 생성 1 회 + 구조화 분석 1 회
```

OpenAI 모드

인물 성향과 단계 과제를 제공하고 이전 발언에 반응하도록 생성

규칙 모드

자료 인용 •확인 필요 문구의 템플릿 / AI 토론으로 표시하지 않음

CODE app/routers/workbench.py

발화를 먼저 보여주고 , 분석을 나중에 합친다

토큰 스트리밍이 아니라 완성된 발화 단위의 SSE 다 .

start | step | analysis | … 반복 … | complete / error

start / step

실행 •인물 초기화 → 대화와 관계망 갱신

analysis

같은 단계 말풍선에 감정 •확실성 • 평판 영향 병합

complete / error

성공한 실행 저장 / 실패 •중단 시 부분 실행 저장하지 않음

CODE app/routers/workbench.py · frontend/src/main.jsx

실제 발언 기록이 마을의 움직임이 된다

Canvas· 프레임 루프 ·BFS 경로 탐색을 대화 단계와 동기화한다 .



24 × 18 타일

카페 · 도서관 · 광장 / 장애물을 피하는 4 방향 경로

발언자 → 청자

두 인물의 만남 •말풍선 •현재 단계 표시

출처 고지

Pixel Agents 이식 코드 •스프라이트 : MIT 고지 유지

CODE frontend/src/VillageScene.jsx · frontend/src/village/town.js

참여 •자료 배정 •변화를 수치로 보여준다

대화 타임라인과 관계망을 함께 읽는다 .



참여

인물별 발언 횟수 / AI 분석 완료 수

범위

검토 자료 배정 수 / 선택 자료 수

흐름

단계별 신뢰도 •문자열 변화 / AI 감정 분포

CODE frontend/src/DiscussionResults.jsx · frontend/src/Comparison.jsx

기초 통계와 AI 의미 분류를 구분한다

같은 분석 함수 •모델 •버전일 때만 AI 수치를 직접 비교한다 .

```
ANALYSIS_VERSION = "text-v1"
client.responses.parse(
    model=analysis_model(),
    text_format=DialogueAnalysis, ...
)
```

기초 지표

표본 수 · 평균 글자 수 · 질문 •전언 표현 비율 / 문자열 규칙

AI 의미 분류

가십 · 감정 · 확실성 · 평판 영향 / DialogueAnalysis 구조화 응답

해석의 경계

신뢰도는 성향 기반 감쇠 / 변화율은 문자열 차이 / 외부 사실 검증 아님

CODE app/services/text_analysis.py · app/models/analysis.py

작은 공개 데모를 위한 사용량 •접근 보호

요청 횟수와 실제 SDK 시도를 별도로 제한한다 .

LLM 예산

IP 당 시간당 10 요청 / 서버 UTC 일일 100 SDK 시도 기본값 / 토큰 •금액 한도 아님

수집 예산

시간당 10 요청 / 일일 외부 HTTP 200 회 / 수집당 최대 10 회

접근 •수명

비공개 자료와 연결 실험 owner 조회 / YouTube 자료 29 일 만료 •연결 실험 삭제

CODE app/services/limits.py · app/services/ai_client.py

핵심 코드와 책임

프로젝트를 설명할 때 바로 열어볼 구현 위치

화면 · 선별

main.jsx: SSE · 화면 전환 / RealDataLab.jsx: 정렬 · 선택 · 비교 · 내보내기

실행 · 분석

workbench.py: 자료 확인 · 인물 배정 · 생성 · 분석 · 저장 / text_analysis.py: 공통 분류 계약

수집 · 보호

collection.py: 고정 공급자 · 출처 서명 · 쿼터 / data_access.py: owner 경계

시각화 · 저장

DiscussionResults.jsx: 지표 · 차트 / VillageScene.jsx: 마을 / runs.py · datasets.py: SQLite

CODE frontend/src/main.jsx · frontend/src/RealDataLab.jsx

확인한 것과 아직 보장하지 않는 것

동작 검증과 모델 성능 검증을 구분한다 .

자동 검증

서버 테스트 34 개 · Node 테스트 10 개 / 임시 DB·HTTP· 모의 AI

브라우저 검증

선별 → 4 인 실행 → SSE → 그래프 → 비교 / 모바일 • 기존 기능 회귀

실서비스 확인

K-HATERS 실제 수집 •저장 ·4 인 규칙 실행 •비공개 비교 •테스트 자료 삭제

한계

실제 OpenAI 응답 품질 •유튜브 키 유효성 •분류 정확도 •대규모 부하를 검증한 수치가 아님

CODE tests/test_discussion.py · tests/browser_discussion.py

데모는 작은 자료 묶음 하나로

실험의 출처 •설정 •결과를 함께 설명한다 .

자료 3 건 저장

정렬 •선택

3~4 명 / 4~8 단계

대화 •자료 배정 확인

비교 •다운로드

필수 설정

수집 : YOUTUBE_API_KEY / AI: OPENAI_API_KEY 또는 ECHOCHAMBER_API_KEY / 선택 : 영속 SQLite 경로

현재 다운로드

CSV: 실제 자료 행 / JSON·HTML: 분석과 선택한 비교 수치 / .ppt·.pptx: 이 발표 자료

다음 확장 제안

명시적 계정 권한 · 사람 평가셋 · 출처 인용 검증 · 토큰 / 비용 예산 · PostgreSQL / 작업 큐

CODE .env.example · render.yaml